

Motor de Búsqueda Comprimido para Secuencias Genómicas usando FM-Index y Wavelet Trees

Joel David Miguel Fernandez – 202310186
Paris Lenard Herrera Torres – 202310100
Ary Werner Aaron Rojas Durand – 202310366

Profesor: Luciano Arnaldo Romero Calla
Profesor: Victor Racso Galvan Oyola

CS3014 – Estructura de Datos Avanzados – UTEC – 2026-1

Repositorio: <https://github.com/Parislht/EDA-PFinal-E1.git>

Link del Video:

<https://drive.google.com/drive/folders/1kt7ju8Q-idUYCKFimvMFMg9wXVZEG646?usp=sharing>

26 de junio de 2026

1. Introducción

La búsqueda de patrones en textos masivos es un problema fundamental en ciencias de la computación con aplicaciones directas en bioinformática, detección de plagio y motores de búsqueda. En el contexto genómico, el problema se materializa en el *alineamiento de lecturas*: dado un genoma de referencia y millones de fragmentos cortos (*reads*) producidos por un secuenciador, se debe determinar en qué posición del genoma aparece cada fragmento. Este proceso es la base para detectar mutaciones asociadas a enfermedades, identificar variantes genéticas y ensamblar genomas nuevos.

El genoma humano contiene aproximadamente $n = 3,2 \times 10^9$ pares de bases del alfabeto $\Sigma = \{A, C, G, T\}$. Un enfoque ingenuo basado en fuerza bruta ($O(nm)$ por consulta) resulta completamente inviable a esta escala. Estructuras clásicas como el *Suffix Array* (Manber & Myers, 1993) permiten búsquedas en $O(m \log n)$, pero exigen almacenar n enteros de 4 bytes, lo que para el genoma humano representa 12.8 GB solo para el arreglo, más 3.2 GB del texto original.

Para resolver simultáneamente los problemas de espacio y velocidad, Ferragina y Manzini (2000) propusieron el **FM-Index**, una estructura que combina la *Burrows-Wheeler Transform* (Burrows & Wheeler, 1994) con tablas de conteo para lograr búsquedas en $O(m \log \sigma)$ sin necesidad de almacenar el texto original ni el Suffix Array completo en memoria. Posteriormente, Grossi, Gupta y Vitter (2003) introdujeron el **Wavelet Tree**, una estructura que permite responder consultas de *rank* sobre alfabetos generales en $O(\log \sigma)$, siendo el complemento ideal para el FM-Index. En este proyecto implementamos el pipeline completo $SA \rightarrow BWT \rightarrow \text{Tabla } C \rightarrow \text{Wavelet Tree}$ y lo evaluamos sobre el genoma de *E. coli* K-12 (4.6 millones de pares de bases).

2. Justificación

Consideremos el genoma humano con $n = 3,2 \times 10^9$ caracteres y un alfabeto de tamaño $\sigma = 4$ (sin contar el centinela \$). Un run típico de secuenciación Illumina produce $\sim 30 \times 10^6$ reads de largo $m = 150$.

Suffix Array clásico. El SA almacena n enteros, cada uno de $\lceil \log_2 n \rceil = 32$ bits:

$$\text{Espacio}_{SA} = n \times 4 \text{ bytes} = 12,8 \text{ GB}$$

Sumando el texto (3,2 GB), el total es ~ 16 GB, lo que excede la RAM de la mayoría de estaciones de trabajo. La búsqueda binaria sobre el SA requiere $O(m \log n)$ comparaciones por query:

$$m \times \log_2 n = 150 \times 31,5 \approx 4,725 \text{ comparaciones de caracteres}$$

Para 30×10^6 queries, esto representa $\sim 1,4 \times 10^{11}$ operaciones.

FM-Index con Wavelet Tree. El Wavelet Tree almacena la BWT en $n \times \lceil \log_2 \sigma \rceil$ bits:

$$\text{Espacio}_{WT} = 3,2 \times 10^9 \times 2 \text{ bits} = 800 \text{ MB}$$

Con un SA sampleado (un valor cada 32 posiciones) para recuperar posiciones:

$$\text{Espacio}_{SA_s} = \frac{n}{32} \times 4 \text{ bytes} = 400 \text{ MB}$$

El espacio total del FM-Index es $\sim 1,2$ GB, una reducción de **13**× respecto al SA clásico. La búsqueda utiliza el *backward search*, que ejecuta m operaciones de *rank*, cada una resuelta en $O(\log \sigma)$ por el Wavelet Tree:

$$m \times \log_2 \sigma = 150 \times 2 = 300 \text{ operaciones de rank}$$

Esto representa una reducción de **15,75**× en operaciones por query frente al SA clásico.

Cuadro 1: Comparación SA clásico vs FM-Index para el genoma humano.

Métrica	SA clásico	FM-Index
Espacio total	~ 16 GB	~ 1.2 GB
Ops. por query ($m=150$)	4,725	300
30M queries (total ops.)	$1,4 \times 10^{11}$	9×10^9

3. Componentes

El programa está implementado en C++20 como un conjunto de clases *header-only* con una cadena clara de dependencias: `BitVector` $\rightarrow$ `WaveletTree` $\rightarrow$ `FMIndex`. El archivo `main.cpp` invoca la construcción completa del índice y ejecuta las consultas. La utilidad `utils.hpp` provee la carga de archivos FASTA y un cronómetro basado en `std::chrono`.

3.1. BitVector

Estructura que almacena una secuencia de bits y responde consultas $\text{rank}_1(i)$ (cantidad de bits en 1 en las posiciones $[0, i)$) en tiempo $O(1)$. Los bits se empaquetan en bloques de 64 bits (`uint64_t`) y se precomputa un arreglo de sumas prefijas por bloque. El conteo de bits dentro de un bloque se realiza con una implementación portátil de `popcount` mediante manipulación de bits, garantizando compatibilidad entre compiladores. También soporta $\text{rank}_0(i) = i - \text{rank}_1(i)$ y `access(i)`.

3.2. WaveletTree

Árbol binario que descompone un string sobre un alfabeto Σ de tamaño σ en $\lceil \log_2 \sigma \rceil$ niveles de bitvectors. En cada nodo interno, el alfabeto se divide en dos mitades: los caracteres con código menor al punto medio se marcan con 0 y los demás con 1. Esto permite resolver $\text{rank}(i, c)$ descendiendo desde la raíz hasta la hoja correspondiente a c , ejecutando una operación de rank_0 o rank_1 en cada nivel. Para el alfabeto genómico $\{\$, A, C, G, T\}$ con $\sigma = 5$, el árbol tiene 4 nodos internos y a lo sumo 3 niveles.

3.3. FMIndex

Clase orquestadora que integra todos los componentes. La construcción sigue el pipeline: (1) Se añade el centinela $\$$ al texto. (2) Se construye el *Suffix Array* mediante *prefix doubling* en $O(n \log^2 n)$, que ordena sufijos comparando pares de rangos $(\text{rank}[i], \text{rank}[i + k])$ con $k = 1, 2, 4, \dots$ (3) Se deriva

la BWT como $BWT[i] = T[SA[i] - 1]$. (4) Se calcula la tabla C , donde $C[c]$ es el número de caracteres en T estrictamente menores que c . (5) Se construye el Wavelet Tree sobre la BWT. La clase expone tres métodos de consulta: `count(P)` retorna el número de ocurrencias, `locate(P)` retorna las posiciones exactas, y `search_verbose(P)` muestra el cálculo paso a paso del *backward search*.

4. Consulta

4.1. Pipeline de búsqueda

El *backward search* lee el patrón $P[0..m-1]$ de derecha a izquierda, manteniendo un rango $[lo, hi)$ en el Suffix Array que acota los sufijos que coinciden con el sufijo del patrón procesado hasta el momento. Inicialmente $lo = 0$ y $hi = n$. En cada iteración, al procesar el carácter $c = P[i]$:

$$lo \leftarrow C[c] + \text{rank}(lo, c), \quad hi \leftarrow C[c] + \text{rank}(hi, c)$$

donde $C[c]$ proviene de la tabla C y `rank` se evalúa sobre el Wavelet Tree de la BWT. Si en algún paso $lo \geq hi$, el patrón no existe en el texto. Al finalizar, el número de ocurrencias es $hi - lo$ y las posiciones se recuperan consultando $SA[lo], \dots, SA[hi-1]$.

4.2. Complejidad

La búsqueda ejecuta exactamente m iteraciones, cada una con dos llamadas a `rank` en el Wavelet Tree ($O(\log \sigma)$ cada una). La complejidad total es $O(m \log \sigma)$. Para el genoma con $\sigma = 5$: $150 \times \lceil \log_2 5 \rceil = 150 \times 3 = 450$ operaciones de `rank` en el peor caso (en la práctica, muchos caminos en el árbol tienen solo 2 niveles). La localización añade $O(occ)$ accesos al SA, donde occ es el número de ocurrencias.

5. Tests y Resultados

5.1. Test 1: Verificación con texto pequeño

Se construyó el FM-Index sobre $T = \text{"GATAGACA"}$ ($n = 9$, incluyendo el símbolo terminal $\$$) con el objetivo de verificar la correctitud de cada una de las estructuras implementadas. El Suffix Array obtenido fue $[8, 7, 5, 3, 1, 6, 4, 0, 2]$, coincidiendo exactamente con el esperado, mientras que la Burrows–Wheeler Transform generada fue $\text{"ACGTGAA\$A"}$, validando la construcción del índice.

La construcción del Suffix Array tomó únicamente **0.00044 s**, la generación de la BWT **0.00057 s** y la construcción del Wavelet Tree **0.00042 s**. Posteriormente se ejecutaron consultas con salida detallada del algoritmo de *backward search*, verificando paso a paso la actualización de los intervalos mediante la función `rank`. Los resultados obtenidos fueron:

- $\text{"GA"} \rightarrow 2$ ocurrencias (posiciones 0 y 4).
- $\text{"TC"} \rightarrow 0$ ocurrencias.
- $\text{"ACA"} \rightarrow 1$ ocurrencia (posición 5).
- $\text{"A"} \rightarrow 4$ ocurrencias (posiciones 1, 3, 5 y 7).

Estos resultados coinciden exactamente con las posiciones reales dentro del texto, validando tanto la construcción del índice como la implementación del algoritmo de búsqueda.

5.2. Test 2: Genoma de *E. coli* K-12

Se evaluó el FM-Index sobre el genoma completo de *Escherichia coli* K-12 MG1655, compuesto por 4 641 652 pares de bases. La construcción del índice se realizó sobre un texto de 4 641 653 caracteres (incluyendo el símbolo terminal $\$$).

Construcción. El Suffix Array requirió **76.85 s**, representando aproximadamente el **98.5 %** del tiempo total de construcción (**78.01 s**). En comparación, la generación de la Burrows–Wheeler Transform tomó únicamente **0.19 s** y la construcción del Wavelet Tree **0.41 s**. Estos resultados muestran claramente que el cuello de botella del sistema continúa siendo la construcción del Suffix Array mediante el algoritmo de *Prefix Doubling* ($O(n \log^2 n)$). El uso de algoritmos lineales como SA-IS permitiría reducir significativamente este tiempo.

Distribución de frecuencias. Las frecuencias obtenidas fueron:

$$A = 1\,142\,742, \quad T = 1\,141\,382, \quad C = 1\,180\,091, \quad G = 1\,177\,437.$$

La distribución cumple las reglas de Chargaff ($A \approx T$ y $C \approx G$), indicando que el genoma fue cargado correctamente.

Consultas individuales. Se verificó el funcionamiento del algoritmo utilizando tanto patrones conocidos como patrones extraídos directamente del genoma. Por ejemplo, “GATTACA” apareció 230 veces, “ACGTACGT” 31 veces y “ATGATGATG” 87 veces. Asimismo, secuencias tomadas exactamente del genoma fueron localizadas en sus posiciones originales (0, 50 000 y 200 000), confirmando la correctitud del procedimiento `locate`.

Benchmark. Finalmente se generaron 100 000 lecturas aleatorias de longitud 150 para evaluar el rendimiento del índice. El tiempo total de búsqueda fue de **3.826 s**, equivalente a un tiempo promedio de únicamente **38.26 μ s** por consulta y un *throughput* de aproximadamente **26 134 consultas por segundo**. Durante el benchmark se localizaron un total de 104 159 ocurrencias. Considerando que una búsqueda por fuerza bruta debería recorrer aproximadamente 4.6 millones de caracteres por consulta, estos resultados evidencian la eficiencia práctica del FM-Index para búsquedas masivas sobre secuencias genómicas.

6. Tareas Pendientes

Para la entrega final se plantean las siguientes tareas:

Genoma humano completo. Escalar las pruebas al genoma de $3,2 \times 10^9$ bases, lo que requerirá optimizaciones de memoria. Este es el benchmark definitivo para validar la viabilidad del FM-Index a escala real.

Optimización del Suffix Array. Reemplazar el algoritmo de prefix doubling $O(n \log^2 n)$ por SA-IS (Nong et al., 2009), que opera en tiempo y espacio $O(n)$, reduciendo la construcción de ~ 52 s a ~ 2 – 3 s para *E. coli*.

SA sampleado. Implementar un Suffix Array sampleado (almacenando una de cada k entradas) para reducir el uso de memoria, recomputando posiciones no muestreadas mediante LF-mapping iterativo.

Demo visual interactiva. Desarrollar una interfaz web con FastAPI/Flask que permita ingresar patrones y visualizar el proceso de backward search, la distribución de ocurrencias sobre el genoma y las estadísticas de rendimiento.

Justificación teórica extendida. Profundizar en la relación formal entre la BWT, la entropía empírica de orden k (Manzini, 2001) y las garantías de compresión del Wavelet Tree, demostrando por qué el FM-Index alcanza espacio proporcional a $nH_k + o(n \log \sigma)$.

7. Conclusiones

En esta primera entrega se implementó exitosamente el pipeline completo del FM-Index en C++20, integrando BitVector con rank en $O(1)$, Wavelet Tree con rank en $O(\log \sigma)$ y el algoritmo de backward search. La implementación fue validada contra valores teóricos conocidos y evaluada sobre un genoma real de 4.6 millones de bases, demostrando un throughput de $\sim 30,000$ queries/segundo con un tiempo promedio de 32.59 μ s por consulta de largo 150. Estos resultados confirman la viabilidad práctica de la estructura: el FM-Index transforma un problema que con fuerza bruta requeriría

recorrer millones de caracteres por consulta en una operación que necesita apenas unos cientos de operaciones de rank. Las tareas pendientes se enfocan en escalar al genoma humano completo y en las optimizaciones necesarias para hacer viable esa escala.

8. Referencias

1. Ferragina, P. y Manzini, G. (2000). Opportunistic data structures with applications. *Proceedings of the 41st Annual Symposium on Foundations of Computer Science (FOCS)*, 390–398. IEEE.
2. Burrows, M. y Wheeler, D. J. (1994). *A block-sorting lossless data compression algorithm* (Technical Report 124). Digital Equipment Corporation, Systems Research Center.
3. Grossi, R., Gupta, A. y Vitter, J. S. (2003). High-order entropy-compressed text indexes. *Proceedings of the 14th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 841–850.
4. Manber, U. y Myers, E. W. (1993). Suffix arrays: A new method for on-line string searches. *SIAM Journal on Computing*, 22(5), 935–948.
5. Nong, G., Zhang, S. y Chan, W. H. (2009). Linear suffix array construction by almost pure induced-sorting. *Proceedings of the Data Compression Conference (DCC)*, 193–202. IEEE.
6. National Center for Biotechnology Information (NCBI). *Escherichia coli str. K-12 substr. MG1655, complete genome*. Acceso: NC_000913.3. https://www.ncbi.nlm.nih.gov/nucleotide/NC_000913.3